

Yuzu-ChatPlatform 开发文档

项目概述

目标

`Yuzu-ChatPlatform` 是一个基于C++、Socket、MySQL编程实现的简单的网络聊天平台，支持用户注册、登录、发送消息以及接收服务器的实时消息。该软件使用了多线程技术，提供了一个基本的客户端和服务端通信框架，并且具有简洁的命令行用户界面。

技术栈

- 编程语言：C++
 - 数据库：MySQL
 - 网络协议：TCP/IP，使用套接字（Socket）进行通信
 - 并发技术：多线程
 - 操作系统：Linux
-

项目结构

- 客户端
 - 客户端（`client.h`, `client.cpp`）
 - 主要用于连接到服务器、发送和接收消息
 - 交互ui（`socialui.h`, `socialui.cpp`）
 - 主要处理客户端的部分加载动画等，提升软件的交互性
- 服务端
 - Server类（`server.h`, `server.cpp`）
 - 负责服务端的基本功能，包括监听端口、接收客户端的连接、处理和转发客户端的消息
 - 提供了客户端的注册（`/signUp`）、登录（`/login`）、消息广播、命令处理等
 - Netizen类（`netizen.h`, `netizen.cpp`）
 - 代表为一个在线的用户。每个Netizen可以有昵称、好友列表、在线状态等信息

- 支持发送消息、发送好友请求、处理好友列表等操作
 - Message类（`message.h`，`message.cpp`）
 - 定义了消息的类型和内容，可以是普通消息或是命令消息
 - DBControl类（`dbcontrol.h`，`dbcontrol.cpp`）
 - 处理数据库相关
 - 用于持久化保存用户数据（如已注册的用户、已添加的好友等）
-

主要功能

1. 用户注册（`/signUp <nickName>`）

- 用户输入自己的网名进行注册，如果网名已经存在，则返回失败
- 调用 `regNetizen` 方法完成注册

2. 用户登录（`/logIn <nickName>`）

- 用户通过自己的网名进行登录。如果网名匹配，则更新用户的在线状态
- 调用 `logNetizen` 方法完成登录

3. 群发消息

- 用户直接在客户端发送消息视为发送普通消息，此时该消息会转发到所有的客户端供所有用户接收

4. 私聊消息（`/m <context>`）

- 用户在客户端可以通过 `/m` 命令后面接上用户名称即可实现私聊发送消息

5. 添加好友（`/friend <nickName>`）

- 用户可以发送好友请求，如果对方同意，则双方互为好友
- 调用 `sendFriendRequest` 方法实现好友请求发送，`acceptFriendRequest` 方法处理对方同意请求

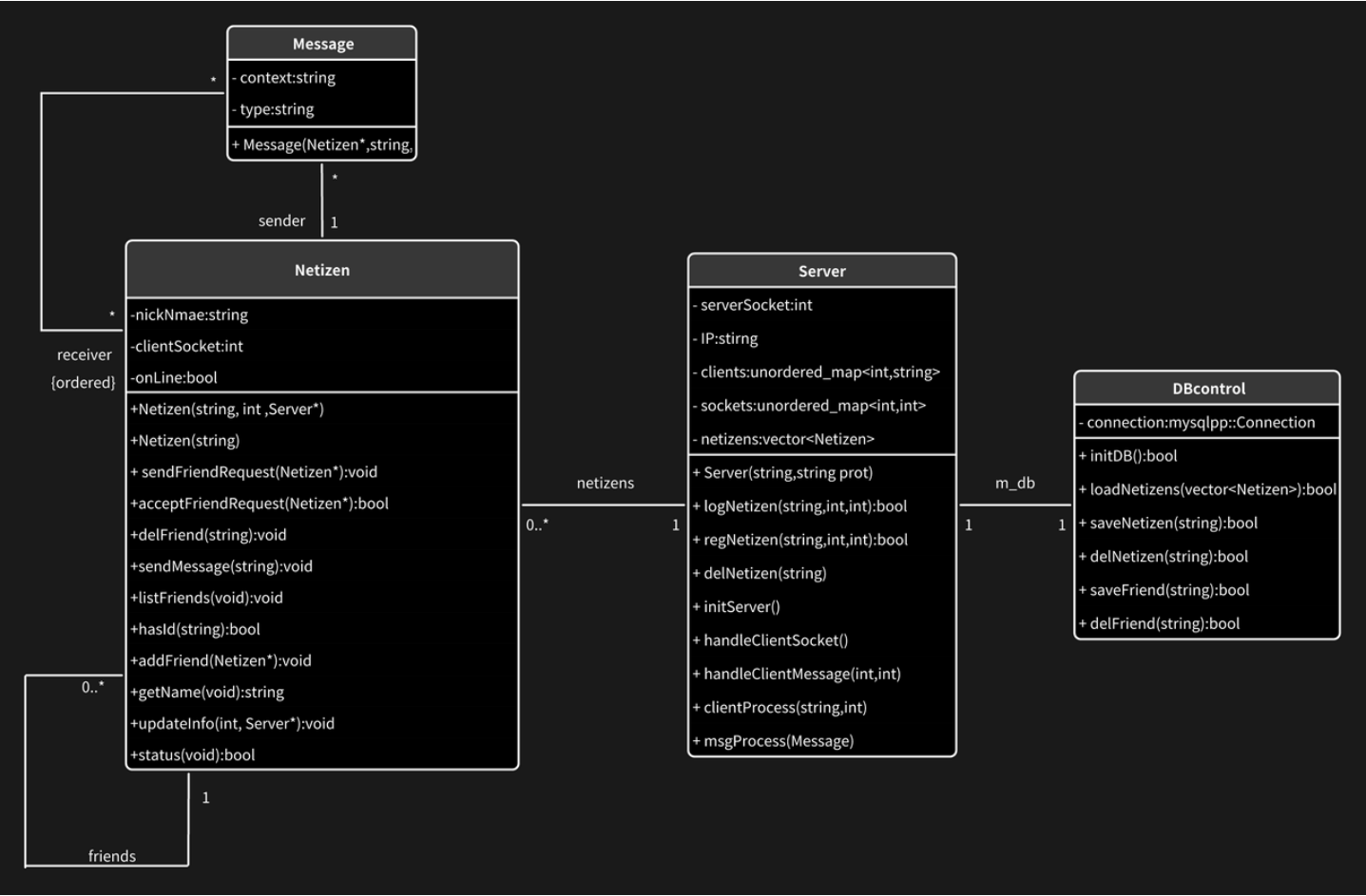
6. 删除好友（`/del <nickName>`）

- 用户可以删除已经添加的好友
- 调用 `delFriend` 删除好友

7. 查看好友列表（`/list`）

- 用户可以查看自己已添加的好友列表
 - 调用 `listFriends` 方法列出所有的好友
-

UML类模型



主要类和方法概述

Server 类

- **initServer:** 初始化服务器，创建套接字、绑定端口、监听请求。
- **handleClientSocket:** 接收客户端连接并为每个客户端创建一个独立线程来处理消息。
- **handleClientMessage:** 处理客户端发送的消息，包括注册、登录、消息发送等。
- **regNetizen:** 注册新用户。
- **logNetizen:** 登录已有用户。
- **delNetizen:** 删除已有用户。
- **clientProcess:** 客户端信息处理。
- **msgProcess:** 处理收到的消息，并根据消息类型进行不同操作。

Netizen 类

- **sendFriendRequest:** 向目标用户发送好友请求。
- **acceptFriendRequest:** 接受好友请求，返回是否同意。

- **delFriend**: 删除好友。
- **sendMessage**: 发送消息到服务器处理，可能是普通消息或命令。
- **listFriends**: 查看好友列表。
- **hasId**: 检查用户是否已经是好友。
- **addFriend**: 导入用户添加好友。
- **updateInfo**: 更新用户的客户端套接字和服务器信息。

DBControl类

- **initDB**: 连接和初始化数据库
 - **loadNetizens**: 导入用户信息
 - **saveNetizen**: 保存指定用户信息
 - **delNetizen**: 删除指定用户信息
 - **saveFriend**: 保存指定用户的好友
 - **delFriend**: 删除指定用户的指定好友
-

代码结构

客户端

```
1 |— client.cpp          # 客户端主程序
2 |— client.h           # 客户端头文件，声明函数接口
3 |— socialui.h         # UI和动画函数
4 |— socialui.cpp       # UI和动画实现
5 |— main.cpp           # 客户端主函数用于启动客户端
```

服务端

```
1 |— server.h           # 服务端头文件，声明类
2 |— server.cpp         # 服务端主程序实现
3 |— netizen.h          # 用户抽象实现声明
4 |— netizen.cpp        # 用户抽象实现
5 |— dbcontrol.h       # 数据库连接头文件
6 |— dbcontrol.cpp     # 数据库连接实现
7 |— message.h         # 消息类抽象
8 |— server.cpp        # 主函数用于启动服务端
```

安装与配置

依赖项

- 编译器: `g++`
- 数据库: `MySQL`
- 依赖库: `MySQL Connector/C++`

源码克隆

```
1 git clone https://github.com/SLSkuy/SocialApp.git
2 # 由于版本提交信息等涉及真实姓名，为保护信息安全，该仓库暂时为私有状态
3 # 库外人员无法通过克隆获取到代码
```

编译代码

若要使用客户端程序则在项目根目录下的client文件夹中打开终端输入以下命令

若要使用服务端程序则在项目根目录下的server文件夹中打开终端输入以下命令

```
1 mkdir build
2 cd build
3 cmake ..
4
5 make
6 sudo make install
```

使用说明

服务端

在终端运行

```
1 YuzuServer
```

服务端启动后根据提示输入是否指定IP等信息之后等待即可

客户端

在终端运行

```
1 YuzuClient
```

程序启动后，显示加载动画，用户需要选择注册（1）或是登录（2）

注册或登录

- 输入用户名并按提示进行注册或登录。
- 如果成功，程序会提示 `signUp successfully` 或 `logIn successfully`。

与服务器通信

- 在登录后，用户可以开始输入消息，并通过 `send` 函数发送到服务器。
- 客户端会通过另一个线程接收来自服务器的消息，并实时显示。

关键技术

- Socket网络编程：使用 POSIX 套接字 API（如 `socket()`，`bind()`，`listen()`，`accept()`，`recv()`，`send()`）进行网络通信。
- 多线程：使用 `std::thread` 来为每一个客户端创建独立的线程，确保能够同时处理多个客户端的请求
- 网络通信规则：规定如何传输消息，传输消息的格式，在客户端将用户发送的消息进行处理后再发往服务端，服务端处理接收的消息以识别用户的意图
- 数据库操作：模拟了数据库的操作（如注册、删除用户、保存用户数据等），通过 `m_db` 对象进行数据库操作

心得

- 团队分工：通过合理的分工以避免项目的开发进度不一致，避免任务重复和疏漏，更快的推动项目开发。
- 团队协作：项目开发过程中需要协作，交换想法和观点，推动项目不断改进和队员的成长
- 项目建模：通过建模合理规划项目的结构、流程，便于团队内部的沟通、理解当前任务和项目目标

- 学无止境：在项目的实现中不断遇到新的问题，学习新的知识并解决问题
-

未来改进

- 消息加密：目前消息的传输处于未加密状态
- 好友验证：由于多线程接收信息的问题，不知道要如何确认对方是否同意添加好友，以及添加好友的指令识别等问题，导致目前的同意添加好友默认为 `true`，后续会为等待结果添加一个状态，阻塞线程知道收到对应的回应以实现好友验证
- 指定建群：目前的群聊为所有人都能参与，后续可以添加指定的几个人为一个群进行群聊聊天
- 图形化界面：后续可以为软件添加图形化交互界面
- 在线状态检测：当前未对在线和离线作区别处理
- 信息收发：在输入时收到信息会导致先前待发送区的信息显示不全
- 历史记录：目前还未实现对于历史信息的存储，后续可以添加会话功能，对每一次会话进行存储，进而实现对于历史聊天记录的查找